CIT651: Introduction to Machine Learning – Assignment 2

Assignment #2 Weight: 25%
The total marks are 100 points

Instructions:

Show all manual calculations and derivations where applicable.
You must use Python (Google Colab or Jupyter Notebook) for all implementations and visualizations.
All interpretations and analyses must be your own work.
Submit a single PDF file along with your Jupyter Notebook (.ipynb) files. No other format will be accepted.

Part 1: Polynomial Regression, Regularization, and Model Analysis (5 points each. 35 points total)

You are provided with two datasets: **heteroscedastic.csv** and **monotonic.csv**. Apply the following steps to both datasets unless otherwise specified.

1. **Exploratory Data Analysis (EDA):** Perform a detailed EDA on both datasets.
   a) Check for missing values, outliers, and understand the distribution of data for each feature.
   b) Generate appropriate visualizations (scatter plots, histograms, box plots).
   c) Apply appropriate data preprocessing techniques based on your EDA findings.

2. **Polynomial Regression - Basic Implementation:** Implement polynomial regression models for both datasets.
   a) Implement polynomial regression of degrees 3 and 4 without any regularization.
   b) Evaluate performance on training and test datasets using RMSE and R^2 score.
   c) Analyze results to show evidence of overfitting. Plot the regression curves and compare performance metrics between training and test sets.

3. **Advanced Regularization Techniques:** Implement regularization to reduce overfitting.
   a) Implement Lasso (L1), Ridge (L2), and Elastic Net regularization for polynomial degrees 3 and 4.
   b) Experiment with different regularization strengths (α) and report the optimal value for each model.
   c) Compare regularized model performance with unregularized models and analyze the impact.

4. Model Optimization and Hyperparameter Tuning: Find the optimal model configuration.

a) Use Grid Search or Random Search to find the optimal hyperparameters (degree and regularization strength) for each dataset.

b) Evaluate the optimized models and compare them with previous models.

5. Handling Heteroscedasticity: (For the Heteroscedastic dataset only)

a) Implement a strategy to handle heteroscedasticity (e.g., transforming the target variable or Weighted Least Squares).

b) Evaluate and compare model performance before and after applying your strategy.

6. Error Analysis and Model Interpretation: Perform a detailed analysis of your best model.

a) Investigate where and why models are making errors, and whether errors follow any patterns (residual analysis).

b) Provide an interpretation of model coefficients and discuss implications in the context of each dataset.

Part 2: Decision Trees and Random Forest (5 points each. 35 points total)

Dataset: Use the **Titanic dataset** from Kaggle (https://www.kaggle.com/competitions/titanic). Your goal is to predict passenger survival.

Data Preprocessing: Handle missing values, encode categorical features (e.g., Sex, Embarked), scale numerical features if needed, and split data into training and validation sets.

Q1. Decision Tree Classifier (Unlimited Depth): Generate an 80/20 split from the training data. Use a Decision Tree with unlimited depth. Average results over 100 random splits and compute the average accuracy.

Q2. Experiment with Depth Limits: Repeat Q1, varying tree depth from 1 to 10. Average results over 100 random splits per depth. Identify the best depth limit for this dataset.

Q3. Overfitting Analysis: Repeat Q2, computing accuracy on both training and validation data. Plot accuracy vs. depth for both sets and analyze overfitting behavior.

Q4. Impact of Training Data Size: Vary the training data size from 5% to 90% in increments of 5%. Use full-depth trees and compute average accuracy for each size. Plot and describe the impact.

Random Forest Classifier:

Q5. Implement a Random Forest Classifier: Train a Random Forest using the same 80/20 split. Compare its performance with the best Decision Tree. Average results over 100 random splits.

Q6. Hyperparameter Tuning for Random Forest: Experiment with different n_estimators and max_depth values. Use 5-fold cross-validation to find the best hyperparameter combination.

Q7. Visualize Random Forest Performance: Plot validation accuracy as a function of n_estimators. Discuss how the number of trees affects performance and overfitting/underfitting trends.

Part 3: K-Means Clustering on CIFAR-10 (5 points each. 25 points total)

Dataset: Use the **CIFAR-10 dataset** (https://www.cs.toronto.edu/~kriz/cifar.html or via TensorFlow/Keras).

Task 1: Load and Explore the Dataset: Load CIFAR-10, display sample images, report the image shape, and the number of images per class.

Task 2: Preprocessing: Normalize pixel values to [0, 1] and flatten images into 1-D feature vectors.

Task 3: Apply K-Means Clustering: Run K-Means with k=10. Report the final inertia and discuss its significance.

Task 4: Visualize Cluster Centroids: Reshape centroids to image format (32x32x3) and visualize them. Label each centroid with its cluster index.

Task 5: Analysis and Discussion: Discuss the limitations of K-Means on CIFAR-10. Explain why pixel-based features may not be suitable for semantic clustering, and suggest improvements.

Part 4: Classification and Model Comparison (Total: 5 points)

Objective: Apply classification models to the **heart disease dataset (heart.csv)**. Implement Logistic Regression from scratch and compare with KNN and SVM.

1. Feature Engineering:

a) Generate new features by combining or transforming existing ones.

b) Apply logarithmic, square root, and polynomial transformations to continuous variables.

c) Evaluate the impact of new features using EDA and validation metrics.

2. Model Implementation:

Logistic Regression (From Scratch): Implement the full algorithm including sigmoid function, loss function (with/without regularization), gradient descent, L1/L2 regularization, prediction and evaluation.

k-Nearest Neighbors (KNN): Use a built-in library. Experiment with different k values and distance metrics (Euclidean and Manhattan).

Support Vector Machines (SVM): Use a built-in library. Experiment with different kernels (linear, polynomial, RBF) and tune hyperparameter C.

3. Evaluation Metrics: For all models, calculate: Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Confusion Matrix. For Logistic Regression, manually compute these metrics.

4. Cross-Validation: Implement k-fold cross-validation for all models. Compare average performance metrics across folds to assess consistency.

Part 4 Evaluation Criteria (Total Marks: 5):

Criteria	Marks
Logistic Regression (From Scratch)	2
KNN and SVM Implementation	1
Evaluation Metrics	1
Report Quality	1

Submission Guidelines

1- Generative AI Policy: Use of generative AI for these problems is strictly prohibited and constitutes academic misconduct as per the course policy [Course Policy section of the provided syllabus].

2- Accuracy: Final answers for numerical results should be rounded to four decimal places. Final answers without supporting details will be marked with ZERO.

3- Deadline: As per the course calendar, late submissions are marked with ZERO.

4- File Format: Submit a single PDF file and all Jupyter Notebook (.ipynb) files. Include code comments and markdown explanations in each notebook.